



01

Statistics Interview QS Set

250+

Interview Questions and
Real Time Scenario Based Questions.

Section D — Database Scenario-Based Coding Questions (3 Questions)

Q13

DATABASE SCENARIO: You have a table called `employee_salaries` with columns: `employee_id`, `department`, `salary`, and `hire_year`. Write SQL queries to compute the Mean, Median (approximate), Mode, and Standard Deviation of salaries — both overall and grouped by department.

✓ SQL Answer

SQL is a bit more limited than Python for statistics, but modern databases have powerful window functions and aggregations. Let me break this into parts.

```
--  
-- PART 1: Overall Summary Statistics  
--  
SELECT  
  COUNT(*) AS total_employees,  
  ROUND(AVG(salary), 2) AS mean_salary,  
  ROUND(STDDEV(salary), 2) AS std_dev_salary,  
  MIN(salary) AS min_salary,  
  MAX(salary) AS max_salary,  
  MAX(salary) - MIN(salary) AS range_salary,  
  ROUND(PERCENTILE_CONT(0.25)  
    WITHIN GROUP (ORDER BY salary), 2) AS q1_salary,  
  ROUND(PERCENTILE_CONT(0.50)  
    WITHIN GROUP (ORDER BY salary), 2) AS median_salary, -- Exact median  
  ROUND(PERCENTILE_CONT(0.75)  
    WITHIN GROUP (ORDER BY salary), 2) AS q3_salary  
FROM employee_salaries;
```

```
--  
-- PART 2: Mode of salary (most frequently occurring value)  
--  
SELECT salary AS mode_salary, COUNT(*) AS frequency  
FROM employee_salaries  
GROUP BY salary  
ORDER BY frequency DESC  
LIMIT 1;
```

```

-- -----
-- PART 3: Department-wise summary with percentiles
-- -----
SELECT
  department,
  COUNT(*) AS headcount,
  ROUND(AVG(salary), 2) AS mean_salary,
  ROUND(PERCENTILE_CONT(0.50)
    WITHIN GROUP (ORDER BY salary), 2) AS median_salary,
  ROUND(STDDEV(salary), 2) AS std_dev,
  ROUND(STDDEV(salary) / AVG(salary) * 100, 2) AS cv_percent, -- Coefficient of
  Variation
  ROUND(PERCENTILE_CONT(0.25)
    WITHIN GROUP (ORDER BY salary), 2) AS q1,
  ROUND(PERCENTILE_CONT(0.75)
    WITHIN GROUP (ORDER BY salary), 2) AS q3
FROM employee_salaries
GROUP BY department
ORDER BY mean_salary DESC;

```

Key notes: PERCENTILE_CONT gives the exact continuous median (interpolates between values). PERCENTILE_DISC gives the nearest actual data value. I always prefer PERCENTILE_CONT for salary analysis. The Coefficient of Variation (CV = SD/Mean × 100%) in the department query lets you compare which departments have the most unequal pay — very useful for HR analysis. Note: PERCENTILE_CONT syntax is standard in PostgreSQL and modern MySQL/MariaDB. In older MySQL, you'd need a workaround with row numbering.

Q14

DATABASE SCENARIO: You have a customer_orders table with columns: order_id, customer_id, product_category, customer_segment ('Premium', 'Standard', 'Basic'), and order_total. Write SQL to build a Contingency Table of customer_segment vs. product_category, include row/column totals, and identify which segment-category combination has the highest purchase frequency.

☒ SQL Answer

Contingency tables in SQL use conditional aggregation — a powerful and often underused technique. Let me build this step by step.

```

-- -----
-- PART 1: Full Contingency Table – Segment vs Category
-- Using conditional aggregation (works in all major databases)
-- -----
SELECT
  customer_segment,
  COUNT(CASE WHEN product_category = 'Electronics' THEN 1 END) AS electronics,
  COUNT(CASE WHEN product_category = 'Clothing' THEN 1 END) AS clothing,

```

```
COUNT(CASE WHEN product_category = 'Groceries' THEN 1 END) AS groceries,  
COUNT(CASE WHEN product_category = 'Books' THEN 1 END) AS books,  
COUNT(*) AS row_total  
FROM customer_orders  
GROUP BY customer_segment
```

```
UNION ALL
```

```
-- Column totals row
```

```
SELECT
```

```
'TOTAL' AS customer_segment,  
COUNT(CASE WHEN product_category = 'Electronics' THEN 1 END),  
COUNT(CASE WHEN product_category = 'Clothing' THEN 1 END),  
COUNT(CASE WHEN product_category = 'Groceries' THEN 1 END),  
COUNT(CASE WHEN product_category = 'Books' THEN 1 END),  
COUNT(*)  
FROM customer_orders  
ORDER BY CASE WHEN customer_segment = 'TOTAL' THEN 1 ELSE 0 END,  
customer_segment;
```

```
-- _____  
-- PART 2: Relative Frequency (proportions within each segment)  
-- _____
```

```
SELECT  
customer_segment,  
product_category,  
COUNT(*) AS freq,  
ROUND(COUNT(*) * 100.0 / SUM(COUNT(*))  
OVER (PARTITION BY customer_segment), 2) AS pct_within_segment,  
ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pct_of_total  
FROM customer_orders  
GROUP BY customer_segment, product_category  
ORDER BY customer_segment, freq DESC;
```

```
-- _____  
-- PART 3: Top segment-category combination by volume  
-- _____
```

```
SELECT customer_segment, product_category, COUNT(*) AS order_count  
FROM customer_orders  
GROUP BY customer_segment, product_category  
ORDER BY order_count DESC  
LIMIT 5;
```

This is a real marketing analytics query. The OVER (PARTITION BY customer_segment) window function computes each row's percentage within its segment — you can see 'what fraction of Premium customers buy Electronics?' Part 3 directly answers 'which combination has the highest frequency'. In practice, this analysis feeds Chi-Square independence testing — if segments and categories are independent, the percentages would be similar across all segments.



02 Linear Algebra Interview QS Set

260+

Interview Questions and
Real Time Scenario Based Questions.

Section C — Coding Questions (3)

Q6. [Coding]

Implement linear regression's closed-form normal equation, including the bias trick (adding a column of ones to X so you don't need a separate intercept term), and use it to predict on new data.

Answer

The 'bias trick' is one of those small things that makes a big difference — without it, the model is forced to fit a hyperplane through the origin, which almost never matches real data. Sticking a column of all 1s onto X gives the model an extra 'always-on' feature, and the weight learned for that column becomes the intercept.

```
import numpy as np

def fit_linear_regression(X, y):
    n = X.shape[0]
    X_aug = np.hstack([X, np.ones((n, 1))]) # bias trick: add column of 1s
    # w* = (X^T X)^-1 X^T y -- but lstsq is more numerically stable than inv()
    w_aug, _, _, _ = np.linalg.lstsq(X_aug, y, rcond=None)
    w, b = w_aug[:-1], w_aug[-1] # split weights and intercept
    return w, b

def predict(X_new, w, b):
    return X_new @ w + b

# Example
X = np.random.randn(100, 3)
y = X @ np.array([2.0, -1.0, 0.5]) + 3.0 + 0.1*np.random.randn(100)

w, b = fit_linear_regression(X, y)
print('weights:', w.round(3), ' intercept:', round(b, 3))

X_new = np.random.randn(5, 3)
preds = predict(X_new, w, b)
print('predictions:', preds.round(3))
```

I used `lstsq` instead of literally inverting $X^T @ X$ — that inverse can blow up numerically if features are correlated, and `lstsq` handles that more gracefully under the hood via SVD.

Q7. [Coding]

Write a function that computes the full pairwise cosine similarity matrix for all rows of a matrix X — no Python for-loops, just broadcasting and matrix multiplication.

Answer

The trick is to first normalize every row to unit length, then just do a single matrix multiplication against its own transpose. Once every row has length 1, the dot product between any two rows IS the cosine similarity directly — no need to divide by norms inside a loop.

```
import numpy as np

def cosine_similarity_matrix(X):
    norms = np.linalg.norm(X, axis=1, keepdims=True) # shape (n,1) -- per-row
    norm
    X_norm = X / norms # broadcasts to (n,d), unit-length rows
    return X_norm @ X_norm.T # (n,d) @ (d,n) -> (n,n) all-pairs

X = np.random.randn(6, 4)
sim_matrix = cosine_similarity_matrix(X)
print(sim_matrix.shape) # (6, 6)
print(np.diag(sim_matrix).round(4)) # should be all 1.0 -- every vector vs
itself
```

Two sanity checks I always run: the diagonal should be exactly 1.0 (every vector is perfectly similar to itself), and the matrix should be symmetric (`sim_matrix[i,j] == sim_matrix[j,i]`) since similarity doesn't care about order.

**03**

Python Interview QS Set

280+

Interview Questions and
Real Time Scenario Based Questions.

PART B — SCENARIO-BASED DESCRIPTIVE QUESTIONS (4)

S1

SCENARIO: You are a data analyst at a food-delivery startup. Your team receives a daily customer list with duplicate user IDs. You also need to find users who placed orders on both Saturday and Sunday. How would you use Python Sets to solve both problems efficiently?

◆ Answer

This is a classic deduplication and overlap-analysis problem — and sets are honestly the perfect tool for exactly this kind of work.

Problem 1 — Removing Duplicate User IDs

Sets automatically enforce uniqueness — any duplicate added to a set is silently ignored. So the cleanest approach is:

```
raw_ids = [1001, 1002, 1001, 1003, 1002, 1004, 1001]
unique_ids = list(set(raw_ids))
print(f"Removed {len(raw_ids) - len(unique_ids)} duplicates")
```

One line. No loop. No conditional logic. The set handles all the deduplication internally.

Problem 2 — Finding Weekend Repeat Customers

Set intersection tells us exactly which elements exist in BOTH sets:

```
saturday_users = {"u101", "u102", "u103", "u104"}
sunday_users = {"u102", "u104", "u105", "u106"}

both_days = saturday_users.intersection(sunday_users)
print("Weekend loyalists:", both_days) # {"u102", "u104"}
```

Why Not Use a Loop or SQL Here?

- A loop would be $O(n \times m)$ complexity. Set intersection is $O(\min(n, m))$ — dramatically faster for large datasets.
- Without a database, sets are the next best thing for this type of filtering.
- I would also use `set.difference()` to find Saturday-only or Sunday-only users for targeted messaging campaigns.

Real bonus: this entire analysis runs before touching pandas, making it ideal for quick pre-processing steps in a data pipeline.

S2

SCENARIO: You work at a fintech company. Your system fetches customer KYC data from an external API page by page (paginated endpoint). Each page returns 100 records as a list of dicts. The API returns an empty list when there are no more pages. Write the logic flow and explain how you would use a while loop to collect all records safely.

◆ Answer

Paginated API fetching is one of the most common real-world while loop patterns. A for loop cannot handle this because we do not know the total number of pages upfront.

Why a While Loop is the Right Tool

- The stopping condition (empty page) is runtime-determined, not pre-known.
- Each iteration depends on the result of the previous one.
- We need a safety guard to prevent infinite loops if the API misbehaves.

The Pattern and Logic

```
all_records = []
page = 1
MAX_PAGES = 50 # safety cap - always include this!

while page <= MAX_PAGES:
    response = fetch_kyc_page(page) # returns list of dicts

    if not response: # empty list = no more data
        print(f"Finished at page {page - 1}")
        break

    all_records.extend(response) # merge into master list
    page += 1

print(f"Total records collected: {len(all_records)}")
```

Three Critical Details

- Use `.extend()`, not `.append()` — `.append()` would create a list of lists instead of a flat list of records.

- Always set `MAX_PAGES` — without it, a buggy API that never returns empty will loop forever and crash your pipeline.
- Check for empty response before processing — avoids index errors on the final empty page.

After collecting `all_records`, one line converts it to a DataFrame: `df = pd.DataFrame(all_records)` — ready for KYC analysis.

**04**

Data Engineering & Code Quality Interview QS Set

260+

Interview Questions and
Real Time Scenario Based Questions.

Level 2: Interviewer Test

10 Questions · 3 Scenario-Based MCQ · 2 Scenario-Based Descriptive · 3 Coding · 2 Database Scenario

Question 11 Scenario-Based MCQ

Scenario: You are building a financial trading dashboard that needs candlestick OHLC data, two moving-average line overlays, and trading volume on a secondary Y-axis — all in one chart. Which approach is correct?

- A) Call `px.line()` three times and combine the resulting figures with separate `fig.show()` calls
- B) Build a `go.Figure()`, add `go.Candlestick` and multiple `go.Scatter` traces, and use a secondary_y axis (via `make_subplots` specs) for the volume bar
- C) This is not possible in Plotly — switch to `matplotlib` for mixed chart types
- D) Use `px.scatter()` with `animation_frame` to cycle through each indicator one at a time

✓ Answer

Correct Answer: B

This is the textbook case for Graph Objects: mixed chart types (candlestick + line + bar) and a secondary Y-axis are both things `px`'s parameter list simply does not expose. The chapter's trading-desk example builds exactly this — roughly 40 lines of `go`, but it is the only way to get all four elements interactive and synchronised in one figure.

Question 12 Scenario-Based MCQ

Scenario: A non-technical business stakeholder wants to interactively switch between viewing "Revenue" and "Margin" on the same chart without you re-running any code. Which Plotly feature directly enables this?

- A) `hovertemplate`
- B) `updatemenus` (dropdown buttons) configured in the layout
- C) `px.colors.qualitative`
- D) `fig.write_image()` with `scale=2`

✓ Answer

Correct Answer: B

update_menus is exactly the dropdown/toggle mechanism described in the chapter's interactivity section — each button in the list can toggle which traces are visible (visible: [True, False, ...]) or restyle the chart entirely. That turns a single static chart into something the stakeholder can self-serve without ever pinging an analyst.

Question 13 Scenario-Based MCQ

Scenario: You need to deliver a chart to a print publication that must look crisp at any zoom level with no pixelation, and the publication does not need any interactivity. Which export format is correct?

- A) `fig.write_html()`
- B) `fig.write_image('chart.png')`
- C) `fig.write_image('chart.svg')`
- D) `fig.to_html(full_html=False)`

✓ Answer

Correct Answer: C

SVG is a vector format — it stores the chart as mathematical shapes rather than a fixed grid of pixels, so it scales to any size or print resolution without pixelating. PNG is raster (it would pixelate if printed large), and HTML formats are for interactive sharing, not static print.

**05**

Exploratory Data Analysis Interview QS Set

200+

Interview Questions and Real Time Scenario Based Questions.

Section C - Coding Questions (3)

Q16. Write a data quality check function that flags rows with an invalid 'age' (outside 0-120) and an unexpected 'gender' value (anything other than 'M' or 'F').

Answer

This is basically automating the schema inspection checklist from the chapter - range checks plus an allowed-category check:

```
import pandas as pd

def check_quality(df):
    invalid_age = df[(df['age'] < 0) | (df['age'] > 120)]
    valid_genders = {'M', 'F'}
    invalid_gender = df[~df['gender'].isin(valid_genders)]
    print('Invalid age rows:', len(invalid_age))
    print('Invalid gender values found:',
          invalid_gender['gender'].unique())
    return invalid_age, invalid_gender

invalid_age_rows, invalid_gender_rows = check_quality(df)
```

The key habit here is returning the actual offending rows, not just a count - so whoever reviews this can immediately go look at those specific records rather than guessing what's wrong. I'd usually wrap this kind of check in an assert or log-and-alert step in a production pipeline so it fails loudly rather than silently.

Q17. Write a function that takes a DataFrame and classifies every column's cardinality into Binary, Low, Medium, High, or Quasi-Unique.

Answer

This is basically packaging the cardinality decision table from the chapter into one reusable function so you're not eyeballing `nunique()` column by column:

```
import pandas as pd
```

```

def classify_cardinality(n_unique, n_rows):
    pct = n_unique / n_rows
    if n_unique == 1:
        return 'Constant - DROP'
    elif n_unique == 2:
        return 'Binary'
    elif n_unique <= 15:
        return 'Low'
    elif n_unique <= 100:
        return 'Medium'
    elif pct > 0.5:
        return 'Quasi-Unique'
    else:
        return 'High'

cardinality_report = pd.DataFrame({
    'unique_values': df.nunique(),
    'total_rows': len(df),
})
cardinality_report['category'] = [
    classify_cardinality(u, len(df)) for u in
cardinality_report['unique_values']
]
print(cardinality_report)

```

The nice thing about writing it this way is that 'category' now directly tells you the encoding strategy: Binary → simple 0/1, Low → one-hot, Medium/High → target/frequency encoding or hashing, Quasi-Unique → drop or extract a feature from it (like email domain).

**06**

Feature Engineering Interview QS Set

220

Interview Questions and
Real Time Scenario Based Questions.

Section D - Database Scenario-Based Coding (2)

Q19. [Database Scenario] Database Scenario: Before deciding which transformation to apply in pandas, you want a quick SQL-level check of a 'transaction_amount' column's skew-related signal - specifically comparing its mean and median directly in the database, since a large gap hints at right skew.

Answer

This mirrors the chapter's mean-vs-median skew heuristic, computed directly in SQL as a cheap first signal before pulling data into pandas for the full skewness/kurtosis check:

```
SELECT
  AVG(transaction_amount) AS mean_amount,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY transaction_amount) AS
  median_amount,
  ROUND(
    (AVG(transaction_amount) - PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY
  transaction_amount))
    / NULLIF(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY
  transaction_amount), 0) * 100, 1
  ) AS pct_gap
FROM transactions;
```

A large positive pct_gap (mean noticeably higher than median) is a strong early hint of right skew - exactly the kind of pattern the chapter's salary example uses to justify reaching for log1p or Box-Cox before the feature ever gets used in a linear model.

Q20. [Database Scenario] Database Scenario: You want to pre-screen which numeric columns in a 'sensor_readings' table are candidates for log transformation versus cube-root transformation, based purely on whether they contain non-positive values - checked directly in SQL before writing any pandas code.

Answer

This mirrors the chapter's 'check min value before choosing a transform' rule, moved upstream into SQL so you know the right transform family before even loading the table:

```

SELECT
  'temperature_change' AS column_name,
  MIN(temperature_change) AS min_value,
  CASE
    WHEN MIN(temperature_change) > 0 THEN 'log1p or sqrt candidate'
    ELSE 'cbirt or Yeo-Johnson required (has zero/negative values)'
  END AS recommendation
FROM sensor_readings
UNION ALL
SELECT
  'defect_count',
  MIN(defect_count),
  CASE
    WHEN MIN(defect_count) > 0 THEN 'log1p or sqrt candidate'
    ELSE 'cbirt or Yeo-Johnson required (has zero/negative values)'
  END
FROM sensor_readings;

```

Running this kind of check across every numeric column before writing any transformation code saves a lot of trial-and-error - the chapter is explicit that `np.log()` and Box-Cox will simply fail (NaN or `ValueError`) the moment they hit a zero or negative value, so knowing the min value upfront tells you immediately which transform family is even viable for each column.

**07**

Data Cleaning & Preprocessing Interview QS Set

120

Interview Questions and Real Time Scenario Based Questions.

Section C - Coding Questions (3)

Q16. Write code to test whether missingness in a column is related to another observed column (checking for MAR), by computing the missingness rate grouped by the other column.

Answer

This is the MAR-detection technique from section 1.1 - checking whether a missingness indicator correlates with another observed variable:

```
import pandas as pd

df = pd.DataFrame({
    'income': [50000, None, 72000, None, 60000],
    'gender': ['M', 'F', 'M', 'F', 'M']
})

df['income_missing'] = df['income'].isnull().astype(int)

print(df.groupby('gender')['income_missing'].mean())
```

If one gender group shows a dramatically higher missingness rate than the other - like the chapter's own 50% vs 0% example - that's strong evidence of MAR, meaning group-wise imputation or a model-based approach would be more appropriate than a single global fill value.

Q17. Write code to apply KNN Imputation correctly, including the required feature scaling step beforehand.

Answer

This is the leakage-aware, scaling-aware KNN workflow from section 1.6 - scaling must come before KNNImputer since it's distance-based:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.impute import KNNImputer
```



```
df = pd.DataFrame({
    'height': [165, 170, np.nan, 180, 175, np.nan],
    'weight': [60, 70, 75, np.nan, 68, np.nan],
    'age': [25, 30, 28, 35, 27, 32]
})

scaler = StandardScaler()
df_scaled = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)

knn_imputer = KNNImputer(n_neighbors=2)
df_imputed_scaled = pd.DataFrame(knn_imputer.fit_transform(df_scaled),
                                  columns=df.columns)

# Convert back to original scale
df_imputed = pd.DataFrame(scaler.inverse_transform(df_imputed_scaled),
                           columns=df.columns)
print(df_imputed.round(1))
```

Skipping the scaling step here would let 'height' (values in the 160-180 range) dominate the distance calculation over any column with a smaller numeric range, silently making the 'nearest neighbours' calculation unreliable.



08

Supervised Learning – Regression Interview QS Set

180+

Interview Questions and
Real Time Scenario Based Questions.

Section B — Scenario-Based Descriptive Questions (2)

Question 14 — Scenario — Sections 1.4/1.7, Durbin-Watson / Independence

Q14 (*Scenario — Sections 1.4/1.7, Durbin-Watson / Independence*)

You compute the Durbin-Watson statistic on the residuals of your regression model and get 0.6.

- (a) If this is a CROSS-SECTIONAL dataset of individual customer loan records (each row = a different, unrelated customer, with no inherent order), how concerned should you be?
- (b) If this is instead a MONTHLY TIME SERIES of total loans disbursed across 36 months, how would your interpretation — and your next steps — change?

💡 Answer & Explanation (Student Walkthrough)

Durbin-Watson around 2.0 means 'no autocorrelation', and a value of 0.6 is well below that, suggesting strong POSITIVE autocorrelation — consecutive residuals are similar to each other. But whether that's actually a problem depends heavily on whether 'consecutive' means anything in your data.

(a) Cross-sectional case: if each row is an independent customer with no real ordering, then 'consecutive residuals' is somewhat arbitrary — it depends on however the rows happen to be sorted in the DataFrame. That said, I wouldn't just shrug this off — a DW of 0.6 here often means the data IS sorted by something correlated with the residuals (e.g., sorted by customer ID, which might correlate with signup date, which might correlate with some unmodeled trend like changing lending policies over time). My first step would be to check how the data is sorted, and maybe re-run DW after shuffling the rows. If shuffling brings DW back near 2, the 'autocorrelation' was really just an artifact of sort order — not a fundamental violation of independence in the statistical sense, but still a clue that there might be a hidden time-based or grouping variable worth investigating.

(b) Time-series case: here, 0.6 is a much bigger deal, because 'consecutive' genuinely means 'this month vs last month' — there IS a real sequence. A DW of 0.6 strongly suggests the model is missing something systematic over time — maybe a trend (loans generally growing over the 36 months) or seasonality (certain months always higher). My next steps would be: add a time-trend feature (e.g., month index) and/or seasonal dummies (month-of-year), check an ACF/PACF plot of the residuals to understand the lag structure, and seriously consider whether a dedicated time-series model (ARIMA/SARIMAX) — or at minimum adding lagged versions of y as features — would be more appropriate than plain OLS for this kind of data.

Question 15 — Scenario — Section 1.8, Reading Coefficients & p-values

Q15 (Scenario — Section 1.8, Reading Coefficients & p-values)

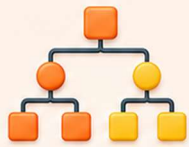
After running `sm.OLS()` for a house price model, you see this in the summary: 'rooms' has coefficient = 3.2, p-value = 0.40; 'area' has coefficient = 0.05, p-value = 0.001. Your stakeholder says: 'Rooms has a much bigger number than area, so rooms must matter more for price — why isn't the model using it more?' How do you respond, and what would you investigate next?

Answer & Explanation (Student Walkthrough)

I'd gently push back on the framing — the size of a coefficient isn't directly comparable across features unless those features are on the SAME SCALE, and 'rooms' and 'area' definitely aren't. 'Rooms' typically ranges from maybe 1 to 5, while 'area' ranges from a few hundred to a few thousand sqft. A coefficient of 3.2 for rooms means '+3.2 per room', while 0.05 for area means '+0.05 per SQUARE FOOT' — over a 1000 sqft range, that's +50, which dwarfs the rooms effect. So the raw numbers aren't telling us 'rooms matters more' at all — they're an artifact of units.

The p-value is actually the more important number here, and it's telling a different story: 'area' has $p=0.001$, meaning we're very confident its effect on price is real and not due to chance. 'rooms' has $p=0.40$, meaning we CAN'T be confident its coefficient differs from zero — statistically, we can't rule out that 'rooms' has no independent effect on price once 'area' is already in the model.

My honest hypothesis here would be multicollinearity — 'rooms' and 'area' are very likely correlated (bigger houses tend to have more rooms), so once 'area' is in the model, 'rooms' might not be adding much NEW information; its effect is partly 'absorbed' by area, which inflates its standard error and pushes the p-value up. What I'd do next: check the correlation between rooms and area (and VIF for both), and consider whether to standardize the features so coefficients ARE directly comparable in magnitude. If VIF confirms strong correlation, I might drop 'rooms' (since 'area' is doing the statistically heavier lifting) or keep both but communicate to the stakeholder that 'rooms' alone doesn't show a reliably independent price effect in THIS model — which is a genuinely useful, if slightly less exciting, finding.

**09**

Supervised Learning – Classification Interview QS Set

200+

Interview Questions and
Real Time Scenario Based Questions.

Q6. [Descriptive]

Explain in your own words why we can't just use plain Linear Regression directly for a binary classification problem like spam detection. What specifically goes wrong?

🎓 How I'd Answer This

Okay, so the core issue is that Linear Regression has no built-in concept of 'this output must stay between 0 and 1.' If I fit $y = w_0 + w_1x_1 + \dots$ on a 0/1 target, the model can happily predict something like -0.4 or 1.8, which just don't make sense as probabilities — you can't be -40% likely to be spam.

There's also a subtler issue: linear regression assumes a constant rate of change everywhere, but real probabilities naturally 'flatten out' near the extremes — going from 50% to 60% confidence should be a lot easier than going from 90% to 99%. Linear regression can't represent that kind of saturation behavior; it just keeps changing linearly forever.

That's exactly why we wrap the linear score z in the sigmoid function instead — it forces the output into (0,1) and naturally captures that diminishing sensitivity near certainty, which matches how probabilities actually behave.

Code:

```
# Linear regression's raw "probability" can go out of bounds
pred = w0 + w1 * x1 # e.g., could be 1.7 -> invalid as a probability
prob = 1 / (1 + np.exp(-pred)) # sigmoid squashes it safely into (0,1)
```

Q7. [Descriptive]

What is an odds ratio in the context of Logistic Regression coefficients, and how do you compute it from a coefficient? Give an example interpretation.

🎓 How I'd Answer This

An odds ratio tells you the multiplicative effect that a one-unit (or one-standard-deviation, if the feature was scaled) increase in a feature has on the odds of the positive class, instead of the probability directly. You get it by exponentiating the raw coefficient: $OR = \exp(\text{coefficient})$.

So say I have a coefficient of 0.74 for 'high blood pressure' — $\exp(0.74) \approx 2.1$, meaning having high blood pressure roughly doubles the odds of a heart attack, holding everything else constant. If the

coefficient were negative instead, say -0.92 , $\exp(-0.92) \approx 0.4$, meaning that feature cuts the odds down to about 40% of baseline, like the 'regular exercise' example from the chapter.

It's a genuinely useful way to explain a model's decision to a non-technical stakeholder or a regulator, since 'this roughly doubles your risk' lands a lot better than reading out a raw log-odds coefficient.

Code:

```
import numpy as np
odds_ratio = np.exp(lr.coef_[0]) # one odds ratio per feature
```

Q8. [Descriptive]

Explain the difference between One-vs-Rest (OvR) and Multinomial (softmax) approaches for multiclass Logistic Regression. Why is multinomial generally theoretically preferred?

 How I'd Answer This

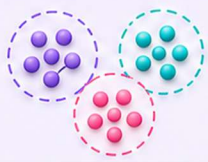
OvR trains one binary classifier per class — for 3 classes, that's 3 separate 'this class vs. everyone else' models, and at prediction time you just go with whichever model was most confident. The catch is those individual probabilities weren't trained together as one coherent system, so they don't naturally sum to 1 — sklearn has to rescale them afterward to make that work.

Multinomial (softmax), on the other hand, trains all classes jointly in a single optimization using the softmax function, so the probabilities are guaranteed to sum to exactly 1 by construction, not by a rescaling trick.

That's basically why multinomial is generally preferred — it's optimizing one coherent joint objective instead of stitching together several independent binary contests, so the resulting probabilities tend to be better calibrated. In sklearn you'd set `multi_class='multinomial'` (or just leave it on 'auto', which picks multinomial automatically when the solver supports it).

Code:

```
lr = LogisticRegression(multi_class="multinomial", solver="lbfgs")
```



10

Unsupervised Learning Interview QS Set

200

Interview Questions and Real Time Scenario Based Questions.

Level 2: Interviewer Test

10 Intermediate/Semi-Advanced Questions — 3 Scenario-Based MCQ, 2 Scenario-Based Descriptive, 3 Coding, 2 Database Scenario (Coding)

Q11. [Scenario-Based MCQ]

Your K-Means model on customer data produces one cluster with 8,000 customers and four other clusters with fewer than 50 customers each. What's the most likely explanation?

- A) K was simply set too low and should be increased
- B) There's likely an outlier or heavily skewed feature distorting distance calculations, or K genuinely doesn't match the data's natural structure; check scaling and consider DBSCAN if cluster shapes are irregular
- C) K-Means always produces unbalanced clusters by design, and this is expected
- D) The random_state simply needs to be changed to a different value

✓ Correct Answer: B

One massive cluster plus several tiny ones is a classic symptom of either unscaled/skewed features distorting distances, or outliers pulling centroids off-center, or a genuine mismatch between K-Means' spherical assumption and the data's real structure — all pointing toward checking scaling first and considering a density-based alternative if shapes are irregular.

Q12. [Scenario-Based MCQ]

You run K-Means with K=5 three times using init='random' and get three different sets of cluster assignments each time. What's the most direct fix for consistency?

- A) This variability is unavoidable and clustering should be abandoned entirely
- B) Use init='k-means++' (the sklearn default) with a sufficiently high n_init, and set random_state for reproducibility
- C) Increase K until the results happen to stabilize
- D) Switch to a completely different distance metric to fix the randomness

✓ Correct Answer: B

Inconsistent results across runs with init='random' is exactly the initialization-sensitivity problem K-Means++ is designed to reduce. Combined with a reasonably high n_init (multiple restarts) and a fixed random_state, this produces far more stable, reproducible results.

Q13. [Scenario-Based MCQ]

You need to cluster GPS coordinates of delivery stops that follow winding roads and have varying numbers of stops per neighborhood (some dense, some sparse). Is K-Means a good fit, and if not, what would you recommend instead?

- A) Yes, K-Means is always the best default choice for geographic data
- B) K-Means is likely a poor fit here since it assumes roughly spherical, similarly-sized clusters; a density-based method like DBSCAN would better handle the irregular, winding shapes and varying density
- C) K-Means requires GPS data to be converted into categorical bins before it can be used at all
- D) This kind of geographic problem cannot be clustered by any algorithm

✓ Correct Answer: B

Winding roads and varying density per neighborhood are exactly the conditions the chapter identifies as poor fits for K-Means' spherical, similarly-sized cluster assumption — DBSCAN's density-based approach handles irregular shapes and varying density far more naturally.

Curriculum of 10 Handbook

104 Chapters Cover • 864 Topics • A Complete Classical ML Journey

STATISTICS

TABLE OF CONTENTS

12 Chapters • 120 Topics • A Complete Statistics Journey

PART 1: Statistics for Data Scientists

#	CHAPTER TITLE	PAGES
1	Chapter 1: Descriptive Statistics (Summarizing & Describing Data) 08 – 26 1.1 Mean, Median & Mode 1.2 Trimmed & Winsorized Mean 1.3 Measures of Spread: Range & Interquartile Range 1.4 Variance & Standard Deviation 1.5 Skewness & Kurtosis 1.6 Percentiles, Quartiles & Deciles 1.7 Z-scores & Standardisation 1.8 Histograms & Box Plots 1.9 Frequency & Contingency Tables 1.10 QQ Plots & Shape Visualisation	
2	Chapter 2: Probability Foundations 2.1 Sample Spaces & Events 2.2 Classical, Empirical & Subjective Probability 2.3 Kolmogorov Axioms 2.4 Addition & Multiplication Rules 2.5 Conditional Probability $P(A B)$ 2.6 Law of Total Probability 2.7 Permutations & Combinations 2.8 Probability Trees & Venn Diagrams 2.9 Expected Value $E(X)$ & Variance $Var(X)$ 2.10 Moment Generating Functions	27 – 51
3	Chapter 3: Probability Distributions 3.1 Bernoulli & Binomial Distributions 3.2 Poisson & Geometric Distributions 3.3 Negative Binomial & Hypergeometric Distributions 3.4 Uniform & Exponential Distributions 3.5 Gamma, Beta & Log-Normal Distributions 3.6 Student's t-Distribution 3.7 Chi-Square & F-Distribution 3.8 Distribution Relationships 3.9 Distribution Selection Framework 3.10 Fitting & Mixture Distributions	52 – 81
4	Chapter 4: Normal Distribution 4.1 Definition & Parameters (μ, σ) 4.2 Standard Normal Distribution $Z \sim N(0,1)$	82 – 111

	4.3 Empirical Rule (68-95-99.7) 4.4 Normal PDF & CDF 4.5 Z-score Interpretation 4.6 Finding Areas Under the Curve 4.7 Inverse Normal (Quantile Function) 4.8 QQ Plots & Shapiro-Wilk Test 4.9 Bivariate & Multivariate Normal Distribution 4.10 Transformations to Achieve Normality	
5	Chapter 5: Central Limit Theorem	112 – 140
	5.1 Law of Large Numbers (Weak & Strong) 5.2 Sampling Distributions 5.3 Standard Error (SE) 5.4 Bias-Variance of Estimators 5.5 CLT Formal Statement 5.6 Proof Intuition via MGFs 5.7 Berry-Esseen Theorem 5.8 CLT for Proportions 5.9 CLT in Practice — Required Sample Size 5.10 Finite Population Correction & Delta Method	
6	Chapter 6: Hypothesis Testing	141 – 174
	6.1 Null & Alternative Hypotheses (H_0, H_1) 6.2 Type I Error (α) — False Positive 6.3 Type II Error (β) — False Negative 6.4 Power ($1-\beta$) & p-value 6.5 One-Sample z-test & t-test 6.6 Independent & Paired Samples t-test 6.7 Chi-Square Goodness of Fit & Independence 6.8 Cohen's d & Effect Size 6.9 Sample Size Calculation 6.10 Bonferroni Correction & FDR	
7	Chapter 7: Confidence Intervals	175 – 202
	7.1 Point vs Interval Estimation 7.2 Correct Interpretation of CIs 7.3 Margin of Error 7.4 CI for Population Mean (z-interval) 7.5 CI for Population Mean (t-interval) 7.6 CI for Proportion 7.7 CI for Variance (Chi-Square) 7.8 CI for Difference of Two Means 7.9 Bootstrap Confidence Intervals 7.10 Bayesian Credible Intervals	
8	Chapter 8: A/B Testing	203 – 241
	8.1 Randomised Controlled Experiments 8.2 Control vs Treatment Groups & Randomisation Chapter 8.3 Minimum Detectable Effect (MDE) 8.4 Sample Size Calculation & Experiment Duration 8.5 Statistical Significance in A/B Tests 8.6 The Peeking Problem 8.7 Sequential Testing & Always-Valid p-values 8.8 Network Effects & Interference 8.9 Multi-Armed Bandit Algorithms 8.10 Bayesian A/B Testing & Multivariate Testing	
9	Chapter 9: Correlation & Regression	242 – 277
	9.1 Pearson, Spearman & Kendall Correlation 9.2 Partial Correlation & Correlation vs Causation 9.3 OLS — Ordinary Least Squares 9.4 R-squared & Adjusted R-squared 9.5 Residual Analysis & Influential Points 9.6 Multiple Linear Regression 9.7 Multicollinearity & VIF 9.8 Regularisation — Ridge & Lasso 9.9 Logistic Regression 9.10 Model Evaluation Metrics	
10	Chapter 10: Bayes Theorem Basics	278 – 307
	10.1 Bayes' Formula Derivation 10.2 Prior, Likelihood & Posterior	

[10.3](#) Bayesian Updating [10.4](#) Bayes vs Frequentist Paradigm
[10.5](#) Beta-Binomial & Gamma-Poisson Conjugacy [10.6](#) Normal-Normal Conjugacy
[10.7](#) Non-Conjugate Priors & MCMC [10.8](#) Naive Bayes Classifier
[10.9](#) Bayesian Hypothesis Testing [10.10](#) Credible Intervals vs Confidence Intervals

11

Chapter 11: Analysis of Variance (ANOVA)

308 – 339

[11.1](#) One-Way ANOVA [11.2](#) F-Statistic & F-Distribution
[11.3](#) Assumptions of ANOVA [11.4](#) Post-Hoc Tests (Tukey & Bonferroni)
[11.5](#) Two-Way ANOVA [11.6](#) Interaction Effects
[11.7](#) Repeated Measures ANOVA [11.8](#) ANCOVA — Analysis of Covariance
[11.9](#) Non-Parametric Alternatives (Kruskal-Wallis) [11.10](#) ANOVA in Python

12

Chapter 12: Time Series Statistics

340 – 378

[12.1](#) Time Series Components (Trend, Seasonality, Noise) [12.2](#) Stationarity & Differencing
[12.3](#) Autocorrelation (ACF) & Partial Autocorrelation (PACF) [12.4](#) Moving Averages & Smoothing
[12.5](#) AR, MA & ARMA Models [12.6](#) ARIMA & Seasonal ARIMA (SARIMA)
[12.7](#) Augmented Dickey-Fuller Test [12.8](#) Exponential Smoothing (Holt-Winters)
[12.9](#) Forecasting Accuracy Metrics [12.10](#) Time Series in Python

MATHEMATICS

TABLE OF CONTENTS

13 Chapters • 104 Topics • A Complete Math for ML Journey

PART 2: Mathematics for Machine Learning

	CHAPTER TITLE	PAGE
1	Chapter 1: Vectors, Matrices & Operations 1.1 What is a Vector 1.2 What is a Matrix 1.3 Vector Operations 1.4 Matrix Operations 1.5 Matrix Multiplication 1.6 Transpose 1.7 Broadcasting in NumPy 1.8 Vectors and Matrices in ML	07-30
2	Chapter 2: Matrix Inverse & Determinant 2.1 What is the Matrix Inverse 2.2 Computing the Inverse 2.3 Identity Matrix 2.4 What is the Determinant 2.5 Computing the Determinant 2.6 Singular Matrices 2.7 Solving Linear Systems 2.8 OLS as a Matrix Equation	31-58
3	Chapter 3: Eigenvalues & Eigenvectors 3.1 What are Eigenvectors 3.2 What are Eigenvalues 3.3 Computing Eigenvalues and Eigenvectors 3.4 Properties of Eigenvalues 3.5 Symmetric Matrices and Real Eigenvalues 3.6 Diagonalisation 3.7 Eigenvalues in PCA 3.8 Eigenvalues in ML	59-87
4	Chapter 4: SVD — Singular Value Decomposition 4.1 What is SVD 4.2 Singular Values 4.3 Left and Right Singular Vectors 4.4 Computing SVD in NumPy 4.5 Truncated SVD 4.6 SVD and PCA 4.7 SVD for Dimensionality Reduction 4.8 SVD in Recommender Systems	88-117
5	Chapter 5: Projections & Orthogonality 5.1 What is a Projection 5.2 Orthogonal Vectors 5.3 Orthogonal Matrices 5.4 Gram-Schmidt Orthogonalisation 5.5 QR Decomposition 5.6 Projection onto a Subspace 5.7 OLS as Projection 5.8 Orthogonality in ML	118-147
6	Chapter 6: Cosine Similarity 6.1 What is Cosine Similarity 6.2 Geometric Intuition	148-178

	6.3 Cosine Similarity vs. Euclidean Distance 6.4 Computing Cosine Similarity in Python 6.5 Cosine Similarity for Text 6.6 Cosine Similarity for Embeddings 6.7 Cosine Similarity Matrix 6.8 Limitations of Cosine Similarity	
7	Chapter 7: Gradient Vectors & Hessian	179-210
	7.1 From Scalar Derivative to Gradient Vector 7.2 The Gradient as Direction of Steepest Ascent 7.3 Computing Gradients in NumPy 7.4 The Jacobian Matrix 7.5 The Hessian Matrix 7.6 Hessian and Convexity 7.7 Newton's Method 7.8 Gradients in ML	
8	Chapter 8: Derivatives & Chain Rule	211-241
	8.1 What is a Derivative 8.2 Basic Differentiation Rules 8.3 Common Derivatives in ML 8.4 The Chain Rule 8.5 Chain Rule Through Multiple Layers 8.6 Symbolic Differentiation with SymPy 8.7 Numerical Differentiation 8.8 Derivatives of Loss Functions	
9	Chapter 9: Partial Derivatives & Gradient	242-266
	9.1 What is a Partial Derivative 9.2 Notation and Computation 9.3 Gradient as Collection of Partial Derivatives 9.4 Directional Derivatives 9.5 Computing Partial Derivatives for ML Loss 9.6 Autograd in PyTorch for Partial Derivatives 9.7 Gradient of Multi-Output Functions 9.8 Partial Derivatives and Feature Sensitivity	
10	Chapter 10: Gradient Descent & Variants	267-294
	10.1 What is Gradient Descent 10.2 The Learning Rate 10.3 Batch Gradient Descent 10.4 Stochastic Gradient Descent (SGD) 10.5 Mini-Batch Gradient Descent 10.6 Momentum 10.7 Adaptive Learning Rate Methods 10.8 Implementing Gradient Descent from Scratch	
11	Chapter 11: Convexity & Loss Functions	295-325
	11.1 What is a Convex Function 11.2 Testing for Convexity 11.3 Convex Optimisation 11.4 Non-Convex Loss Surfaces 11.5 MSE Loss 11.6 Cross-Entropy Loss 11.7 Regularised Loss Functions 11.8 Loss Function Selection Guide	
12	Chapter 12: Backpropagation Intuition	326-355
	12.1 What Backpropagation Computes 12.2 Computation Graphs 12.3 Forward Pass Through a Computation Graph 12.4 Backward Pass 12.5 Backprop for a Single Neuron 12.6 Backprop for a Two-Layer Network 12.7 Vanishing and Exploding Gradients 12.8 How Autograd Implements Backprop	
13	Chapter 13: Numerical Optimisation Methods	356-385
	13.1 Why Numerical Optimisation 13.2 scipy.optimize Overview 13.3 Gradient-Free Optimisation 13.4 L-BFGS 13.5 Constrained Optimisation 13.6 Global Optimisation 13.7 Hyperparameter Optimisation as Numerical Optimisation 13.8 When to Use Which Method	

PYTHON

PART 01

TABLE OF CONTENTS

14 Chapters • 112 Topics • A Complete Python for Data Science Journey

PART 1: Python for Data Science

CHAPTER TITLE	PAGE
1 Chapter 1: Python Fundamentals	08-18
1.1 Variables, Constants, and PEP 8 Naming Conventions 1.2 Primitive Data Types: String, Integer, Float, Boolean 1.3 String Manipulation: f-strings, .strip(), .split(), .replace() 1.4 Type Casting and Checking (type(), isinstance()) 1.5 Mathematical Operators & Order of Operations 1.6 Logical Operators (and, or, not) and Membership (in) 1.7 Conditional Logic: if, elif, else 1.8 Comparison Operators for Data Filtering	
2 Chapter 2: Data Structures & Iteration (The Logic)	19-33
2.1 Lists: Slicing, appending, and list methods 2.2 Dictionaries: Key-value pairs for JSON-like data 2.3 Sets & Tuples: Unique values and immutability 2.4 For Loops: Iterating through lists and dictionaries 2.5 While Loops: Logic-based iteration 2.6 List Comprehensions: Writing “Pythonic” one-liners for data transformation 2.7 Dictionary Comprehensions: Transforming mapping data 2.8 Nested structures: Lists of dictionaries (the standard API format)	
3 Chapter 3: Functions & Error Handling	34-52
3.1 Defining Functions: def, arguments, and return 3.2 Positional vs. Keyword arguments 3.3 Lambda Functions: Anonymous functions for quick data mapping 3.4 Scope: Local vs. Global variables 3.5 Error Handling: try, except, finally to prevent script crashes 3.6 Importing modules (import math, import os) 3.7 Understanding __name__ == “__main__” 3.8 Documentation: Writing Docstrings for collaborative code	
4 Chapter 4: NumPy — The Foundation of Data	53-71
4.1 The ndarray: Why it’s faster than Python lists 4.2 Array creation: np.array(), np.zeros(), np.arange(), np.linspace() 4.3 Vectorization: Performing math on entire columns without loops 4.4 Broadcasting: How NumPy handles different array shapes 4.5 Universal Functions (ufuncs): np.exp(), np.sqrt(), np.log() 4.6 Slicing and Masking: Filtering data using Boolean arrays 4.7 Aggregations: np.sum(), np.mean(), np.std(), np.median() 4.8 Reshaping and Transposing data	
5 Chapter 5: pandas — Core DataFrame Operations	72-94

[5.1](#) Series vs. DataFrames: The 1D and 2D structures [5.2](#) Data I/O: Reading and writing CSV, Excel, and JSON
[5.3](#) Inspection: `.head()`, `.info()`, `.describe()`, `.shape` [5.4](#) Indexing: Selecting data with `.loc` (labels) and `.iloc` (positions)
[5.5](#) Conditional Filtering: Selecting rows based on multiple criteria [5.6](#) Column Operations: Renaming, dropping, and creating new features
[5.7](#) Sorting: `.sort_values()` and `.sort_index()` [5.8](#) Setting and Resetting Indexes

6 Chapter 6: pandas — Advanced Wrangling 95-116

[6.1](#) Merging: `pd.merge()` (Inner, Left, Right, Outer joins) [6.2](#) Concatenation: `pd.concat()` (Stacking data vertically or horizontally)
[6.3](#) Groupby: Split-Apply-Combine logic [6.4](#) Aggregation: `.agg(['mean', 'sum', 'count'])`
[6.5](#) Pivot Tables and Cross-tabulations (`pd.pivot_table()`) [6.6](#) Transform & Filter: Advanced Groupby operations
[6.7](#) Multi-indexing: Handling complex, hierarchical data [6.8](#) Apply: Using custom functions on rows/columns with `.apply()` and `.map()`

7 Chapter 7: The Data Cleaning Protocol 117-138

[7.1](#) Identifying and handling Missing Data: `.isnull()`, `.dropna()`, `.fillna()` [7.2](#) Imputation strategies: Mean, Median, and Mode vs. Forward-fill
[7.3](#) Data Type Conversion: `.astype()` and `pd.to_datetime()` [7.4](#) Handling Duplicates: `.duplicated()` and `.drop_duplicates()`
[7.5](#) String Cleaning in DataFrames: `.str` accessor methods [7.6](#) Outlier Detection: Z-score and IQR (Interquartile Range) methods
[7.7](#) Regex for Data Cleaning: Using `re` module to fix messy patterns [7.8](#) Standardizing columns and category labels

8 Chapter 8: Data Collection (APIs & Scraping) 139-162

[8.1](#) The requests library: GET and POST requests [8.2](#) JSON Parsing: Converting API responses into DataFrames
[8.3](#) Authentication: Working with API Keys and Headers [8.4](#) Web Scraping Basics: Introduction to BeautifulSoup
[8.5](#) Inspecting HTML: Tags, Classes, and IDs [8.6](#) Scraping tables and lists into pandas
[8.7](#) Handling Pagination: Scraping across multiple pages [8.8](#) Ethical Scraping: `robots.txt` and rate limiting

9 Chapter 9: Static Visualization (Matplotlib & Seaborn) 163-187

[9.1](#) The Object-Oriented Interface: `fig`, `ax = plt.subplots()` [9.2](#) Customizing Plots: Titles, Labels, Legends, and Colors
[9.3](#) Basic Plots: Line, Bar, Scatter, and Pie [9.4](#) Statistical Viz (Seaborn): `sns.histplot`, `sns.kdeplot`, `sns.boxplot`
[9.5](#) Categorical Viz: `sns.violinplot`, `sns.stripplot`, `sns.countplot` [9.6](#) Regression Plots: `sns.regplot` and `sns.lmplot`
[9.7](#) Relationship Grids: `sns.pairplot` and `sns.heatmap` [9.8](#) Visualization Best Practices: Data-ink ratio and choosing the right chart

10 Chapter 10: Interactive Viz & Dashboards (Plotly & Streamlit) 188-215

[10.1](#) Interactive Scatter and Line plots with Plotly Express [10.2](#) Adding Interactivity: Hover data, Sliders, and Buttons
[10.3](#) Geographic Maps: Choropleths and Scatter Geo [10.4](#) Streamlit Fundamentals: `st.write()`, `st.sidebar`, `st.columns`
[10.5](#) Input Widgets: Selectboxes, Sliders, and File Uploaders [10.6](#) Integrating pandas with Streamlit for live filtering
[10.7](#) Displaying charts and DataFrames in a web app [10.8](#) Deploying a Python Dashboard to the Cloud

11 Chapter 11: Exploratory Data Analysis (EDA) & Stats 216-239

11.1 Descriptive Statistics: Central tendency and dispersion **11.2** Skewness and Kurtosis: Understanding distribution shapes
11.3 Correlation: Pearson vs. Spearman coefficients **11.4** Hypothesis Testing: Null vs. Alternative hypothesis
11.5 T-Tests: One-sample, Independent, and Paired **11.6** ANOVA: Comparing means across three or more groups
11.7 Chi-Square Test for Independence (Categorical data) **11.8** P-values and Significance levels ($\alpha = 0.05$)

12

Chapter 12: Predictive Modeling (Regression)

240-265

12.1 Introduction to Scikit-Learn: The fit-predict API **12.2** Simple Linear Regression: OLS (Ordinary Least Squares)
12.3 Multiple Linear Regression: Handling multiple features **12.4** Assumptions of Linear Regression (Normality, Homoscedasticity)
12.5 Logistic Regression: Sigmoid function and binary classification **12.6** Model Evaluation (Regression): R^2 , MAE, MSE, RMSE
12.7 Model Evaluation (Classification): Confusion Matrix, Precision, Recall, F1-Score **12.8** Train-Test Split and Cross-Validation

13

Chapter 13: Time Series Analysis

266-290

13.1 Handling Datetime objects: `pd.to_datetime()` and `.dt` accessor **13.2** Resampling and Frequency Conversion (`.resample('M')`, `.asfreq()`)
13.3 Window Functions: Rolling means and Expanding windows **13.4** Time Shifting: `.shift()` and `.diff()` for growth rates
13.5 Seasonality and Trend decomposition **13.6** Handling Time Zones and Daylight Savings
13.7 Visualizing Time Series: Seasonal subseries plots **13.8** Basic Forecasting: Autocorrelation (ACF) and Moving Averages

14

Chapter 14: Unsupervised Learning (Clustering)

291-317

14.1 Introduction to Unsupervised Learning vs. Supervised **14.2** K-Means Clustering: How the algorithm works
14.3 Choosing K: The Elbow Method and Silhouette Score **14.4** Data Scaling: Why `StandardScaler` is required for clustering
14.5 Principal Component Analysis (PCA) for Dimensionality Reduction **14.6** Interpreting Cluster Centroids and Labels
14.7 Hierarchical Clustering (Dendrograms) **14.8** Real-world Case Study: Customer Segmentation

PYTHON

PART 02 & 03

TABLE OF CONTENTS

13 Chapters • 108 Topics • A Complete Python for Data Science Journey

CHAPTER TITLE	PAGE
1 Chapter 1: SciPy — Scientific Computing	07-25
1.1 What is SciPy and How It Relates to NumPy 1.2 scipy.stats — Statistical Functions 1.3 Hypothesis Testing with scipy.stats 1.4 scipy.optimize 1.5 scipy.interpolate 1.6 scipy.spatial 1.7 scipy.signal 1.8 When to Use SciPy vs. sklearn vs. statsmodels	
2 Chapter 2: Plotly — Advanced Interactive Viz	26-46
2.1 Plotly Express Recap and Scope 2.2 Plotly Graph Objects 2.3 Subplots and Multi-Panel Figures 2.4 Advanced Interactivity 2.5 Custom Colour Scales and Themes 2.6 3D Visualisations 2.7 Plotly for ML Visualisation 2.8 Exporting Plotly Figures	
3 Chapter 3: FastAPI & API Integration	47-71
3.1 From requests to FastAPI 3.2 FastAPI Fundamentals 3.3 Building GET and POST Endpoints 3.4 Pydantic Models for Data Validation 3.5 Consuming External APIs in DS Workflows 3.6 Authentication Patterns 3.7 Async API Calls 3.8 Integrating APIs into a DS Pipeline	
4 Chapter 4: Advanced Python for DS	72-97
4.1 Object-Oriented Programming for DS 4.2 Generators and Iterators 4.3 Decorators for DS Code 4.4 Context Managers 4.5 Python Performance Profiling 4.6 Multiprocessing and Parallelism 4.7 Working with Paths and Files 4.8 Environment Variables and Config Management	
PART 3: Data Engineering & Code Quality	
1 Chapter 1: SQL in Python	98-120
1.1 Why SQL in Python 1.2 SQLite for Local Development 1.3 SQLAlchemy — Database Abstraction Layer 1.4 pandas read_sql and read_sql_query	

[1.5](#) Writing DataFrames to SQL [1.6](#) psycopg2 for PostgreSQL

[1.7](#) SQL Query Optimisation for DS [1.8](#) Parameterised Queries and SQL Injection Prevention

2 Chapter 2: NoSQL — MongoDB Basics 121-144

[2.1](#) What is NoSQL and When to Use It [2.2](#) MongoDB Concepts

[2.3](#) Connecting with pymongo [2.4](#) CRUD Operations

[2.5](#) Querying with Filters [2.6](#) MongoDB Aggregation Pipeline

[2.7](#) Converting MongoDB Results to pandas [2.8](#) When MongoDB Over SQL for DS

3 Chapter 3: Cloud Data — BigQuery 145-169

[3.1](#) What is BigQuery [3.2](#) BigQuery Setup

[3.3](#) google-cloud-bigquery Library [3.4](#) Reading BigQuery Results into pandas

[3.5](#) Writing DataFrames to BigQuery [3.6](#) BigQuery SQL Dialect

[3.7](#) Cost Management in BigQuery [3.8](#) BigQuery vs. Local SQL vs. DuckDB

4 Chapter 4: Parquet & Arrow 170-193

[4.1](#) Why Parquet [4.2](#) Reading and Writing Parquet with pandas

[4.3](#) Apache Arrow [4.4](#) pyarrow Basics

[4.5](#) Parquet Partitioning [4.6](#) Parquet vs. CSV vs. Feather

[4.7](#) Reading Parquet from Cloud Storage [4.8](#) Arrow for High-Performance DS Pipelines

5 Chapter 5: DuckDB 194-217

[5.1](#) What is DuckDB [5.2](#) Setting Up DuckDB

[5.3](#) Running SQL on pandas DataFrames [5.4](#) Running SQL on CSV and Parquet Files

[5.5](#) DuckDB vs. pandas for Aggregations [5.6](#) Reading Results into pandas

[5.7](#) DuckDB with Cloud Storage [5.8](#) DuckDB vs. SQLite vs. BigQuery

6 Chapter 6: Python Packaging 218-239

[6.1](#) Why Packaging Matters for DS [6.2](#) Project Structure

[6.3](#) pyproject.toml [6.4](#) pip install -e .

[6.5](#) Dependency Management [6.6](#) Virtual Environments Recap

[6.7](#) Publishing to PyPI [6.8](#) Internal Packages for DS Teams

7 Chapter 7: Testing ML Code 240-265

[7.1](#) Why Testing ML Code is Different [7.2](#) pytest Fundamentals

[7.3](#) Testing Data Pipelines [7.4](#) Testing Preprocessing Functions

[7.5](#) Fixtures in pytest [7.6](#) Mocking External Dependencies

[7.7](#) Testing Model Outputs [7.8](#) Continuous Testing in CI

8 Chapter 8: Logging & Error Handling 266-291

[8.1](#) Beyond try/except [8.2](#) Python logging Module

[8.3](#) Log Levels [8.4](#) Logging to Files and Console Simultaneously

[8.5](#) Structured Logging with structlog [8.6](#) Exception Handling Patterns for DS

[8.7](#) Logging in ML Pipelines [8.8](#) Common Logging Mistakes

9 Chapter 9: Type Hints, Docstrings & Linting 292-316

[9.1](#) Why Code Quality Tools Matter for DS [9.2](#) Type Hints Basics
[9.3](#) Type Hints for DS Functions [9.4](#) mypy for Static Type Checking
[9.5](#) Google-Style Docstrings [9.6](#) ruff for Linting and Formatting
[9.7](#) Pre-commit Hooks [9.8](#) Applying Code Quality to a DS Project

EDA & FEATURE ENGINEERING

PART 01

TABLE OF CONTENTS

10 Chapters • 80 Topics • A Complete EDA Journey

PART 1: Exploratory Data Analysis

	CHAPTER TITLE	PAGE
1	Chapter 1: Data Types & Feature Understanding 1.1 Types of Variables 1.2 Structured vs. Unstructured Data 1.3 Statistical Treatment by Variable 1.4 Identifying Variable 1.5 Data Schema Inspection 1.6 Type Mismatches and Corrections 1.7 Cardinality Assessment 1.8 Building a Variable Inventory	07-39
2	Chapter 2: Data Quality Assessment 2.1 What is Data Quality 2.2 Detecting Duplicate Rows 2.3 Inconsistent Formatting 2.4 Erroneous and Impossible Values 2.5 Structural Errors in Categorical Data 2.6 Mixed Data Types in a Single Column 2.7 Data Quality Scoring 2.8 Data Quality Report as an EDA Artifact	40-77
3	Chapter 3: Automated EDA Profiles 3.1 Introduction to Automated EDA 3.2 Overview of EDA Profiling Libraries 3.3 Generating a Profile Report 3.4 Dataset Statistics Summary 3.5 Variable Type Detection 3.6 Correlation and Interaction Overview 3.7 Alerts and Quality Warnings 3.8 Customizing and Exporting Reports	78-115
4	Chapter 4: Univariate Analysis 4.1 What is Univariate Analysis? 4.2 Descriptive Statistics 4.3 Distribution Shape Measures 4.4 Visualizing Numeric Distributions 4.5 Visualizing Categorical Variables 4.6 Analyzing Text and Date Variables 4.7 Normality Testing 4.8 Transformations for Distribution Correction	116-152
5	Chapter 5: Bivariate Analysis 5.1 Introduction to Bivariate Analysis 5.2 Numeric vs. Numeric Relationships 5.3 Categorical vs. Categorical Relationships 5.4 Numeric vs. Categorical Relationships 5.5 Correlation Heatmaps 5.6 Feature–Target Relationship Analysis	153-190

[5.7](#) Interaction Effects Between Features [5.8](#) Statistical Significance Testing

6	Chapter 6: Multivariate Analysis	191-230
	6.1 From Bivariate to Multivariate Thinking 6.2 Pair Plots and Scatter Matrix 6.3 Dimensionality Reduction — PCA 6.4 Dimensionality Reduction — t-SNE and UMAP 6.5 Cluster Analysis for EDA 6.6 Parallel Coordinates and Radar Charts 6.7 Feature Importance from Tree Models 6.8 VIF and Multicollinearity Detection	
7	Chapter 7: Missing Data Analysis	231-272
	7.1 Types of Missingness 7.2 Quantifying Missing Data 7.3 Visualizing Missingness Patterns 7.4 Testing for Non-Random Missingness 7.5 Simple Imputation Strategies 7.6 Advanced Imputation Methods 7.7 Newton's Method 7.8 Evaluating Imputation Quality	
8	Chapter 8: Outlier Detection	273-310
	8.1 What is an Outlier? 8.2 Visual Outlier Detection 8.3 Statistical Outlier Methods 8.4 Distance-Based Methods 8.5 Isolation Forest and Ensemble Methods 8.6 Multivariate Outlier Detection 8.7 Outlier Treatment Strategies 8.8 Outliers in Context of the Target Variable	
9	Chapter 9: Class Imbalance Analysis	311-349
	9.1 Understanding Class Imbalance 9.2 Measuring and Visualizing Imbalance 9.3 Why Accuracy is a Misleading Metric 9.4 Undersampling Techniques 9.5 Oversampling Techniques 9.6 Combined and Ensemble Resampling 9.7 Cost-Sensitive Learning 9.8 Evaluation Metrics for Imbalanced Data	
10	Chapter 10: Target Leakage Detection	350-389
	10.1 What is Target Leakage? 10.2 Train-Test Leakage 10.3 Temporal Leakage 10.4 Group Leakage 10.5 Leakage Detection via Feature Importance 10.6 Correlation-Based Leakage Screening 10.7 Preventing Leakage with Pipelines 10.8 Cross-Validation Strategies for Leakage Prevention	

EDA & FEATURE ENGINEERING

PART 02

TABLE OF CONTENTS

11 Chapters • 88 Topics • A Complete Feature Engineering Journey

PART 2: Feature Engineering

	CHAPTER TITLE	PAGE
1	Chapter 1: Numerical Transformations	07-48
	1.1 Introduction to Numerical Transformations 1.2 Log Transformation	
	1.3 Square Root and Cube Root Transformations 1.4 Box-Cox Transformation	
	1.5 Yeo-Johnson Transformation 1.6 Reciprocal and Power Transformations	
	1.7 Quantile Transformation 1.8 Evaluating Transformation Effectiveness	
2	Chapter 2: Encoding Categorical Variables	49-85
	2.1 Introduction to Categorical Encoding 2.2 Label Encoding	
	2.3 Ordinal Encoding 2.4 One-Hot Encoding	
	2.5 Binary Encoding 2.6 Frequency and Count Encoding	
	2.7 Hashing Encoding (Feature Hashing) 2.8 Handling Rare and Unknown Categories	
3	Chapter 3: Target Encoding	86-127
	3.1 What is Target Encoding 3.2 Mean Target Encoding	
	3.3 Target Leakage Risk in Target Encoding 3.4 Cross-Validation-Based Target Encoding	
	3.5 Smoothing and Regularisation 3.6 Leave-One-Out Encoding	
	3.7 Target Encoding for Regression vs. Classification 3.8 Evaluating Target Encoding Quality	
4	Chapter 4: Feature Scaling & Normalisation	128-167
	4.1 Why Feature Scaling Matters 4.2 Min-Max Scaling (Normalisation)	
	4.3 Standardisation (Z-score Scaling) 4.4 Robust Scaling	
	4.5 Max Abs Scaling 4.6 Unit Vector Scaling (Normaliser)	
	4.7 Scaling in Pipelines 4.8 Choosing the Right Scaler	
5	Chapter 5: Date/Time Feature Extraction	168-207
	5.1 Introduction to Temporal Features 5.2 Basic Datetime Decomposition	
	5.3 Cyclical Encoding of Temporal Features 5.4 Derived Time Gap Features	
	5.5 Business Calendar Features 5.6 Time-Series Lag and Rolling Features	

[5.7 Handling Timezones and Irregular Timestamps](#) [5.8 Feature Engineering for Event-Based Data](#)

6	Chapter 6: Interaction & Polynomial Features	208-249
	6.1 What are Interaction Features 6.2 Manual Interaction Feature Creation 6.3 Polynomial Feature Expansion 6.4 Dimensionality Explosion Problem 6.5 Interaction Features for Tree vs. Linear Models 6.6 Ratio and Difference Features 6.7 Feature Selection After Interaction Expansion 6.8 Evaluating the Value of Interaction Features	
7	Chapter 7: Binning & Discretisation	250-291
	7.1 Introduction to Binning 7.2 Equal-Width Binning 7.3 Equal-Frequency (Quantile) Binning 7.4 Custom Domain-Driven Binning 7.5 Optimal Binning 7.6 Encoding Bins as Features 7.7 Binning for Outlier Handling and Robustness 7.8 When to Bin and When Not To	
8	Chapter 8: Aggregation Features	292-305
	8.1 Introduction to Aggregation Features 8.2 GroupBy Aggregations 8.3 Target-Based Aggregations 8.4 Ratio and Deviation Features 8.5 Aggregations Across Multiple Granularities 8.6 Aggregation in Time-Series Context 8.7 Aggregation for Relational/Multi-Table Data 8.8 Feature Explosion Management	
9	Chapter 9: High Cardinality Features	306-319
	9.1 Defining High Cardinality 9.2 Identifying High Cardinality Columns 9.3 Frequency-Based Grouping 9.4 Supervised Cardinality Reduction 9.5 Hashing Tricks for High Cardinality 9.6 Embedding-Based Encoding for High Cardinality 9.7 High Cardinality in Tree vs. Linear Models 9.8 Evaluating Cardinality Reduction Impact	
10	Chapter 10: Domain Knowledge Feature Creation	320-333
	10.1 Why Domain Knowledge Matters in Feature Engineering 10.2 Collaborating with Domain Experts 10.3 Ratio and Business Metric Features 10.4 Threshold and Rule-Based Features 10.5 Composite Score Features 10.6 Domain-Driven Aggregation 10.7 Validating Domain Features Statistically 10.8 Documenting Domain Features	
11	Chapter 11: Feature Selection	334-347
	11.1 Why Feature Selection is Essential 11.2 Filter Methods 11.3 Wrapper Methods 11.4 Embedded Methods 11.5 Feature Importance from Tree Models 11.6 Multicollinearity-Based Selection 11.7 Stability and Cross-Validated Feature Selection 11.8 Building a Final Feature Set	

EDA & FEATURE ENGINEERING PART 03

TABLE OF CONTENTS

6 Chapters • 48 Topics • A Complete Data Cleaning Journey

PART 3: Data Cleaning & Pre-processing

	CHAPTER TITLE	PAGE
1	Chapter 1: Handling Missing Values	06-33
	1.1 Types of Missingness 1.2 Detecting and Quantifying Missing 1.3 Visualising Missingness Patterns 1.4 Dropping Missing Data 1.5 Simple Imputation 1.6 Advanced Imputation 1.7 Missingness Indicator Features 1.8 Imputation Inside a Pipeline	
2	Chapter 2: Handling Outliers	34-49
	2.1 Types of Outliers 2.2 Visual Detection 2.3 Statistical Detection Methods 2.4 Model-Based Detection 2.5 Multivariate Outlier Detection 2.6 Outlier Treatment 2.7 Outliers in Context of the Target 2.8 Outlier Handling Inside a Pipeline	
3	Chapter 3: Handling Inconsistent & Erroneous Data	50-64
	3.1 What is Inconsistent Data 3.2 Case and Whitespace Standardisation 3.3 Fixing Categorical Inconsistencies 3.4 Detecting and Correcting Erroneous Values 3.5 Regex-Based Cleaning 3.6 Standardising Date and Numeric Formats 3.7 Cross-Column Consistency Checks 3.8 Automating Inconsistency Detection	
4	Chapter 4: Handling Duplicates	65-79
	4.1 What are Duplicate Records 4.2 Detecting Exact Duplicates 4.3 Key-Based Duplicate Detection 4.4 Near-Duplicate Detection 4.5 Strategies for Removing Duplicates 4.6 Aggregating Instead of Dropping 4.7 Duplicates in Time-Series Data 4.8 Documenting and Auditing Duplicate Removal	
5	Chapter 5: Data Type Conversion & Formatting	80-95
	5.1 Why Data Types Matter 5.2 Inspecting Data Types 5.3 Numeric Type Conversion 5.4 Categorical Type Conversion 5.5 Datetime Parsing and Conversion 5.6 Boolean Type Handling 5.7 Downcasting for Memory Optimisation 5.8 Building a Type Correction Function	
6	Chapter 6: sklearn Pipeline	96-111
	6.1 What is a Pipeline 6.2 Why Pipelines Prevent Leakage	

[6.3 Building a Simple Pipeline](#) [6.4 Column Transformer](#)

[6.5 Custom Transformers](#) [6.6 Pipeline with Feature Selection](#)

[6.7 Cross-Validation with Pipeline](#) [6.8 Saving and Deploying Pipelines](#)

CLASSICAL MACHINE LEARNING

PART 01

TABLE OF CONTENTS

9 Chapters • 72 Topics • A Complete Classical ML Journey

PART 1: Classical Machine Learning — Regression

1	Chapter 1: Linear Regression (OLS)	03-14
	1.1 What is Linear Regression 1.2 Mathematical Intuition 1.3 Ordinary Least Squares (OLS) 1.4 Assumptions of Linear Regression 1.5 Simple Linear Regression in Python 1.6 Multiple Linear Regression 1.7 Checking and Validating Assumptions 1.8 Interpreting Model Output	
2	Chapter 2: Ridge & Lasso Regression	15-26
	2.1 Why Regularisation is Needed 2.2 What is Regularisation 2.3 Ridge Regression (L2) 2.4 Lasso Regression (L1) 2.5 ElasticNet Regression 2.6 Tuning the Alpha Hyperparameter 2.7 Comparing OLS vs. Ridge vs. Lasso 2.8 When to Use Ridge vs. Lasso	
3	Chapter 3: Polynomial Regression	27-38
	3.1 Limitations of Linear Regression for Curved Data 3.2 What is Polynomial Regression 3.3 PolynomialFeatures in sklearn 3.4 Fitting a Polynomial Regression Model 3.5 Choosing the Right Polynomial Degree 3.6 Bias-Variance Trade-off in Polynomial Regression 3.7 Cross-Validation for Degree Selection 3.8 Limitations and Practical Cautions	
4	Chapter 4: Decision Tree Regressor	39-50
	4.1 Introduction to Decision Trees for Regression 4.2 How a Regression Tree Splits 4.3 Tree Structure Components 4.4 Building a Decision Tree Regressor in sklearn 4.5 Key Hyperparameters 4.6 Overfitting in Decision Trees 4.7 Pre-Pruning Strategies 4.8 Feature Importance in Decision Trees	
5	Chapter 5: Random Forest Regressor	51-62
	5.1 From Decision Trees to Ensembles 5.2 What is Bagging 5.3 How Random Forest Works 5.4 Building a Random Forest Regressor in sklearn 5.5 Key Hyperparameters 5.6 Feature Importance in Random Forest 5.7 Hyperparameter Tuning with GridSearchCV 5.8 Advantages and Limitations	
6	Chapter 6: Gradient Boosting — XGBoost Regressor	63-74

[6.1](#) From Bagging to Boosting [6.2](#) Gradient Boosting Intuition
[6.3](#) What Makes XGBoost Different [6.4](#) Installing and Setting Up XGBoost
[6.5](#) Key Hyperparameters [6.6](#) Early Stopping
[6.7](#) Feature Importance in XGBoost [6.8](#) Hyperparameter Tuning for XGBoost

7 Chapter 7: Gradient Boosting — LightGBM Regressor 75-86

[7.1](#) What is LightGBM and Why It Exists [7.2](#) Leaf-Wise vs. Level-Wise Tree Growth
[7.3](#) GOSS and EFB [7.4](#) Setting Up LightGBM
[7.5](#) Key Hyperparameters [7.6](#) Categorical Feature Handling
[7.7](#) Early Stopping and Callbacks in LightGBM [7.8](#) XGBoost vs. LightGBM

8 Chapter 8: CatBoost Regressor 87-98

[8.1](#) What is CatBoost and Why It Was Built [8.2](#) Ordered Boosting
[8.3](#) Native Categorical Feature Handling [8.4](#) Setting Up CatBoost
[8.5](#) Key Hyperparameters [8.6](#) Handling Categorical Features in Practice
[8.7](#) Overfitting Detection in CatBoost [8.8](#) XGBoost vs. LightGBM vs. CatBoost

9 Chapter 9: SVR — Support Vector Regression 99-110

[9.1](#) From SVM Classification to Regression [9.2](#) The Epsilon-Insensitive Tube
[9.3](#) Support Vectors in Regression [9.4](#) Kernel Functions
[9.5](#) Key Hyperparameters [9.6](#) Feature Scaling Requirement
[9.7](#) Building SVR in sklearn [9.8](#) When to Use SVR

CLASSICAL MACHINE LEARNING

PART 02

TABLE OF CONTENTS

10 Chapters • 80 Topics • A Complete Classical ML Journey

PART 2: Classical Machine Learning — Classification

1	Chapter 1: Logistic Regression	07-30
	1.1 From Linear to Logistic Regression 1.2 The Sigmoid Function 1.3 Decision Boundary 1.4 Binary vs. Multiclass Logistic Regression 1.5 Building Logistic Regression in sklearn 1.6 Regularisation in Logistic Regression 1.7 Interpreting Coefficients 1.8 Assumptions and Limitations	
2	Chapter 2: Naive Bayes	31-53
	2.1 Introduction to Probabilistic Classification 2.2 Bayes Theorem 2.3 The Naive Independence Assumption 2.4 Types of Naive Bayes 2.5 Building Naive Bayes in sklearn 2.6 Text Classification with Naive Bayes 2.7 Laplace Smoothing 2.8 When to Use Naive Bayes	
3	Chapter 3: K-Nearest Neighbours (KNN)	54-78
	3.1 Introduction to Instance-Based Learning 3.2 How KNN Works 3.3 Distance Metrics 3.4 Choosing the Right K 3.5 Building KNN in sklearn 3.6 Feature Scaling Requirement 3.7 Curse of Dimensionality in KNN 3.8 Strengths and Limitations	
4	Chapter 4: Decision Trees Classifier	79-104
	4.1 How Decision Trees Classify 4.2 Splitting Criteria 4.3 Tree Structure 4.4 Building a Decision Tree Classifier in sklearn 4.5 Overfitting and Pruning 4.6 Key Hyperparameters 4.7 Feature Importance 4.8 Advantages and Limitations	
5	Chapter 5: Random Forest Classifier	105-129
	5.1 Ensemble Learning Recap 5.2 How Random Forest Classification Works 5.3 Building Random Forest Classifier in sklearn 5.4 Key Hyperparameters 5.5 Handling Class Imbalance 5.6 Feature Importance in Random Forest 5.7 Out-of-Bag (OOB) Score 5.8 Hyperparameter Tuning with GridSearchCV	
6	Chapter 6: XGBoost Classifier	130-147

[6.1](#) XGBoost for Classification [6.2](#) Binary vs. Multiclass Classification
[6.3](#) Building XGBoost Classifier [6.4](#) Key Hyperparameters
[6.5](#) Handling Class Imbalance with `scale_pos_weight` [6.6](#) Early Stopping in Classification
[6.7](#) SHAP Values for XGBoost Feature Importance [6.8](#) XGBoost Classification Tuning Strategy

7 Chapter 7: CatBoost Classifier 148-166

[7.1](#) CatBoost for Classification [7.2](#) Handling Categorical Features in Classification
[7.3](#) Building CatBoost Classifier [7.4](#) Key Hyperparameters
[7.5](#) Auto Class Weights [7.6](#) Overfitting Detection
[7.7](#) Visualising CatBoost Training [7.8](#) CatBoost vs. XGBoost vs. LightGBM for Classification

8 Chapter 8: LightGBM Classifier 167-185

[8.1](#) LightGBM for Classification [8.2](#) Binary and Multiclass Classification Setup
[8.3](#) Building LightGBM Classifier [8.4](#) Key Hyperparameters
[8.5](#) Handling Class Imbalance [8.6](#) Native Categorical Support in Classification
[8.7](#) Early Stopping and Evaluation Metrics [8.8](#) LightGBM on Large Datasets

9 Chapter 9: SVM — Support Vector Machine Classifier 186-203

[9.1](#) Core Idea of SVM [9.2](#) Support Vectors and Margins
[9.3](#) The Kernel Trick [9.4](#) C Parameter
[9.5](#) Gamma Parameter [9.6](#) Building SVM Classifier in sklearn
[9.7](#) Feature Scaling Requirement [9.8](#) When to Use SVM

10 Chapter 10: Stacking & Voting Ensembles 204-222

[10.1](#) Introduction to Ensemble Methods [10.2](#) Hard Voting vs. Soft Voting
[10.3](#) Building a Voting Classifier in sklearn [10.4](#) Introduction to Stacking
[10.5](#) Building a Stacking Classifier in sklearn [10.6](#) Choosing Base Models for Ensembles
[10.7](#) Avoiding Leakage in Stacking [10.8](#) When Ensembles Help and When They Do Not

CLASSICAL MACHINE LEARNING

PART 03

TABLE OF CONTENTS

6 Chapters • 48 Topics • A Complete Unsupervised ML Journey

PART 3: Classical Machine Learning — Unsupervised Learning

- 1 Chapter 1: K-Means Clustering..... 06–23**
1.1 What is Clustering 1.2 How K-Means Works 1.3 Choosing K 1.4 K-Means++ Initialisation 1.5 Building K-Means in sklearn 1.6 Evaluating Clustering Quality 1.7 Limitations of K-Means 1.8 Practical Applications
- 2 Chapter 2: Hierarchical Clustering..... 24–41**
2.1 What is Hierarchical Clustering 2.2 Agglomerative vs. Divisive 2.3 Linkage Methods 2.4 Dendrogram 2.5 Building Hierarchical Clustering in sklearn 2.6 Choosing Number of Clusters 2.7 Comparing K-Means vs. Hierarchical 2.8 Use Cases
- 3 Chapter 3: DBSCAN..... 42–59**
3.1 Limitations of K-Means and Hierarchical 3.2 How DBSCAN Works 3.3 Key Parameters 3.4 Building DBSCAN in sklearn 3.5 Choosing Epsilon and Min_samples 3.6 Strengths of DBSCAN 3.7 Limitations of DBSCAN 3.8 DBSCAN for Anomaly Detection
- 4 Chapter 4: t-SNE & UMAP..... 60–78**
4.1 Why PCA is Not Enough 4.2 What is t-SNE 4.3 Key Parameters in t-SNE 4.4 Building t-SNE in sklearn 4.5 What is UMAP 4.6 Key Parameters in UMAP 4.7 Building UMAP in Python 4.8 t-SNE vs. UMAP vs. PCA
- 5 Chapter 5: Anomaly Detection.....79–97**
5.1 What is Anomaly Detection 5.2 Types of Anomalies 5.3 Statistical Methods 5.4 Isolation Forest 5.5 Local Outlier Factor (LOF) 5.6 Building Isolation Forest and LOF 5.7 Evaluating Anomaly Detection 5.8 Real-World Applications
- 6 Chapter 6: Association Rules.....98–116**
6.1 What is Association Rule Mining 6.2 Key Concepts 6.3 The Apriori Algorithm 6.4 The FP-Growth Algorithm 6.5 Building Association Rules with mlxtend 6.6 Interpreting Rules 6.7 Setting Thresholds 6.8 Real-World Applications